

Quant Singularity – AI-SLM Signal Pod

Technical Report · AI Research Engineer Intern Screening · Summer 2026

Kaggle Notebook	https://www.kaggle.com/code/safalsingh/quant-singularity-finetune-and-eval
Base Model	TinyLlama/TinyLlama-1.1B-Chat-v1.0
MLflow Run (final)	f0a11051b56d44e790bb9d1ad3aef18c
Training	Kaggle T4 GPU · 3 epochs · LoRA rank=8 · val_loss=0.974
No-RAG accuracy	0.2949 · FAIL (threshold ≥ 0.45)
RAG accuracy	0.3742 · RAG HELPS (+7.93%)

1. Eval Suite Design

This section was written and committed (eval_suite.py) before the first Kaggle training run.

1.1 Metrics and Conditions

The following metrics define whether the pod is trustworthy enough to connect to the orchestrator. All threshold values were committed before any training result was observed.

Metric	Definition	Pass Threshold	Fail Threshold
Schema pass rate	Fraction of outputs that parse as valid JSON with all required fields and correct types	≥ 0.95	< 0.95
Global directional accuracy	Fraction of active (non-suppressed) signals matching ground truth direction	≥ 0.45	< 0.38
Per-window accuracy	Accuracy within each 5-day rolling block	≥ 0.38	< 0.30 triggers review

Metric	Definition	Pass Threshold	Fail Threshold
Conviction ECE	Expected Calibration Error between conviction and realised accuracy	≤ 0.20	> 0.30
Parse failure rate	Fraction of calls returning NEUTRAL due to unparseable output	≤ 0.02	> 0.05
ADX suppression rate	Fraction of rows suppressed by ADX < 20 rule	Reported, not thresholded	—
Conviction downgrade rate	Fraction of signals downgraded to NEUTRAL by conviction < 0.40	≤ 0.45	> 0.60

Random baseline for 3-class classification: **0.333**. A model that cannot reliably beat random is not production-safe.

1.2 Walk-Forward Protocol

Evaluation uses strict temporal ordering: days 1–30 for training, days 31–60 for evaluation, assessed in six non-overlapping 5-day blocks. K-fold cross-validation on a time series is disqualifying because it leaks future regime information into training.

1.3 Conviction Validity – Design Problem

Conviction is not a softmax probability. The model generates the conviction value as text – a floating point number between 0.0 and 1.0. For this value to be meaningful it must be *calibrated*: a model claiming conviction 0.80 should be correct ~80% of the time.

Calibration is assessed via:

1. **ECE (Expected Calibration Error)** – binned accuracy vs mean conviction
2. **Reliability diagram** – visual binned comparison
3. **Conviction distribution check** – if the model always outputs > 0.90 , it is not using the full scale

A conviction field that clusters near 1.0 regardless of market conditions is not a quality gate – it is a hallucination. The threshold (0.40) only functions if the model generates below-threshold values for genuinely uncertain states.

1.4 Regime-Sliced Evaluation

VIX is split at its median value across the eval window. Accuracy is computed separately for high-VIX ($\geq$ median) and low-VIX ($<$ median) subsets. A robust model should not degrade catastrophically in high-VIX

conditions. A model that only works in calm markets is not safe.

2. Data Audit & Cleaning

2.1 Dataset Overview

The raw instruction dataset(`finetune_instructions.jsonl`) contained **300 training samples** in Alpaca-style format. Each sample encodes a market state snapshot as `input` and a structured JSON trading signal as `output` . The `market_states.parquet` file contained 780 rows across 60 trading days (Oct-Dec 2024), with labels: `NEUTRAL` (316), `PE` (239), `CE` (225).

2.2 Conviction Field Corruption – Primary Finding

Inspection of the raw instruction file revealed that rows **~48-92** originated from a different data pipeline that serialised the `conviction` field as human-readable text instead of a float:

Type	Count	Examples
Valid numeric float	255	<code>0.47</code> , <code>0.52</code> , <code>0.61</code>
Correctable string	6	<code>"0.8 (high)"</code> → <code>0.80</code>
Uncorrectable string	39	<code>"high"</code> , <code>"moderate"</code> , <code>"low"</code> , <code>"strong"</code>

Decision: The 39 uncorrectable rows were **excluded** from training. Including them would teach the model to produce schema-violating `conviction` values, directly breaking the orchestrator's JSON parser. The 6 correctable rows were **patched** by regex-extracting the leading float and re-validating the range `[0.0, 1.0]` . This is conservative: when in doubt, exclude rather than impute.

Final clean dataset: 261 rows (255 original + 6 patched, 39 excluded).

2.3 Other Findings

- **ATM IV floor (37 rows):** Values clamped to 10.0 – the NSE minimum IV floor. These are legitimate market states, not data errors. Kept as-is.
- **Invalid JSON:** Zero rows – all 300 rows parsed cleanly as JSON.
- **Training label distribution (clean set):** `NEUTRAL` =116, `CE` =75, `PE` =70. Mild class imbalance with `NEUTRAL` over-represented (~44%).

2.4 Audit Methodology

The audit script (`data_audit.py`) was committed to the repository **before** any training run. It produces `finetune_instructions_clean.jsonl` and `audit_report.txt` deterministically.

3. Fine-Tuning and RAG

3.1 Base Model Selection: TinyLlama-1.1B-Chat-v1.0

Criterion	TinyLlama-1.1B	Phi-2 (2.7B)
VRAM (fp16 training)	~2.2 GB	~5.4 GB
Fits Kaggle T4 (15 GB)	✔ Comfortably	✔ Tight
Chat instruction-tuning	✔ Pre-tuned	Partial
Overfitting risk at 261 samples	Lower (fewer params)	Higher
4-bit inference on CPU	< 2s/call	~5s/call

The `chat-v1.0` checkpoint is already instruction-tuned on a large conversation corpus, providing a strong base for Alpaca-style SFT. Phi-2 would be preferred with >1000 samples where its larger capacity adds value. At 261 samples, TinyLlama's smaller footprint is safer.

3.2 Instruction Template with Worked Example

Prompt format (Alpaca-style):

```
### Instruction:
You are a NIFTY 50 options signal generator. Return ONLY valid JSON with
direction (CE/PE/NEUTRAL), conviction (float 0-1), horizon (intraday/next
signal_id (string), generated_at (string).

### Input:
{"nifty_spot": 22859.61, "atm_iv": 13.41, "iv_skew_25d": 3.88, "pcr": 1.1
"adx_14": 29.35, "realized_vol_5d": 13.6, "vix_india": 14.08,
"dte_nearest": 2, "moneyness_band": "ATM"}

### Response:
```

Expected output (from training label):

```
{
  "direction": "CE",
  "conviction": 0.63,
  "horizon": "next_session",
  "signal_id": "3f8a1b2c-9d4e-4f5a-8b6c-7d2e1f0a9b3c",
  "generated_at": "2024-10-15T09:30:00+05:30"
}
```

Feature interpretation for this example: ADX=29.35 (strong trend, above suppression threshold), VIX=14.08 (calm), PCR=1.13 (put-heavy, CE contrarian signal), DTE=2 (expiry close), skew=3.88 (puts bid). Label CE with conviction 0.63 reflects moderate directional confidence in a put-selling regime.

3.3 LoRA Configuration

Parameter	Value	Rationale
Rank (<code>r</code>)	8	Balanced for 261 samples: rank=4 under-fits, rank=16 over-fits for a narrow 3-class task
Alpha	16	2× rank — standard scaling, stable gradient flow
Dropout	0.05	Light regularisation on small data
Target modules	<code>q_proj</code> , <code>v_proj</code>	Query and value projections; standard for Llama-2 architecture
Trainable params	1,126,400 (0.10%)	Minimal adapter over frozen base

Rank 8 rationale: The task is narrow-domain structured generation from 9 features to a 3-class JSON schema. The base model already understands JSON and number generation — LoRA needs to shift the output distribution toward market-domain labels, not learn new capabilities. Rank 8 provides sufficient capacity for this shift without memorising the 235 training samples.

3.4 Training Results

Epoch	Train Loss	Val Loss
1	1.741	1.613
2	1.167	1.098
3	0.972	0.974

Val-train gap = 0.002 at epoch 3 — no overfitting. `load_best_model_at_end=True` saved epoch 3 as the final adapter.

3.5 Conviction Field – Design

Conviction is generated as a **learned numeric text target**. The model is trained on examples where conviction values were assigned by the original data pipeline (mean=0.495, std=0.107 in the clean set). Through SFT, the model learns to emit a float that correlates with the features supporting the labelled direction.

Why softmax is wrong: `softmax(logits)[direction_token]` gives `P("CE" | prompt)` – the probability that the string "CE" follows the prompt, not a calibrated estimate of whether the underlying market regime supports a CE signal. Softmax confidence reflects the model's linguistic certainty, not the signal quality. A model can be very confident it should type "CE" while being wrong about the market.

What makes conviction meaningful (ideally): The training data pairs market states with conviction values proportional to feature alignment – a state with ADX=32, VIX=12, PCR=1.2 all consistent with a CE regime should get higher conviction than a state with mixed signals. Through SFT, the model internalises this mapping.

Observed limitation: ECE=0.5586 in final eval – the conviction field was not meaningfully calibrated. The model learned to emit high conviction (mean=0.854) uniformly, regardless of actual accuracy. Fix: Platt scaling on a calibration holdout, or raising the conviction threshold to ≥ 0.75 .

3.6 MLflow Run Table

Run ID	Epochs	Val Loss	Notes
2c8edc45...	3	0.969	First clean run
6437e9ed...	5	1.151	5-epoch test – worse, not selected
f0a11051...	3	0.974	Final submission
6437e9ed...	3	–	Balanced ablation (CE=312 collapse)

3.7 RAG Experiment

Prompt template with retrieved context:

```
### Instruction:
You are a NIFTY 50 options signal generator. Use the historical episodes
below to ground your signal. Return ONLY valid JSON.

Similar historical episodes:
- Regime: trending_up | ADX=28.1, VIX=15.2, PCR=1.13 | Outcome: CE
```

- Regime: neutral | ADX=19.8, VIX=14.0, PCR=0.97 | Outcome: NEUTRAL
- Regime: trending_down | ADX=31.2, VIX=22.5, PCR=0.85 | Outcome: PE

Input:
{<current market state as JSON>}

Response:

Retrieval: Top-3 episodes from rag_corpus.jsonl via provided retrieve(market_state, k=3) – KNN over normalised ADX, VIX, skew, PCR, DTE features. Function was not modified.

Ablation results:

Metric	No-RAG	With RAG	Delta
Global accuracy	0.2949	0.3742	+0.0793
95% CI	(0.247, 0.348)	(0.322, 0.429)	—
CE predicted	234	95	-139
PE predicted	78	106	+28
NEUTRAL predicted	78	189	+111

RAG HELPS: +7.93% accuracy gain. Retrieved context dramatically changed prediction distribution – NEUTRAL predictions jumped from 78 to 189, much closer to ground truth NEUTRAL=151. RAG partially corrected the CE directional bias by grounding the model in historical episodes where PE and NEUTRAL outcomes were observed. Conviction increased slightly with RAG (0.854 → 0.934), which may reflect the model's increased text confidence when it has corroborating context – not necessarily a calibration improvement.

4. Results

4.1 No-RAG Summary (Primary Condition)

Metric	Result	Threshold	Status
Schema pass rate	1.000	≥ 0.95	✓ Pass
Parse failure rate	0.0%	≤ 2%	✓ Pass
Parse recovered (regex fallback)	3.6% (14/390)	—	✓ Working
ADX suppression (< 20)	20.0% (78/390)	—	✓ Deterministic

Metric	Result	Threshold	Status
Conviction downgrade (< 0.40)	0.0%	≤ 45%	✅ Pass
Mean conviction	0.8535	—	⚠️ Overconfident
Conviction ECE	0.5586	≤ 0.20	❌ Fail
Global directional accuracy	0.2949 CI=(0.247, 0.348)	≥ 0.45	❌ Fail
High-VIX accuracy (VIX ≥ 21.4)	0.2735 CI=(0.201, 0.361)	—	—
Low-VIX accuracy (VIX < 21.4)	0.3077 CI=(0.247, 0.376)	—	—

4.2 Per-Window Results

Window	Accuracy	95% CI	n Active	ADX Suppressed	Flag
2024-11-12 – 2024-11-18	0.3231	[0.222, 0.444]	65	0	❌
2024-11-19 – 2024-11-25	0.3846	[0.276, 0.506]	65	0	✅
2024-11-26 – 2024-12-02	0.2308	[0.082, 0.503]	13	52	❌
2024-12-03 – 2024-12-09	0.2308	[0.127, 0.383]	39	26	❌
2024-12-10 – 2024-12-16	0.2615	[0.170, 0.380]	65	0	❌
2024-12-17 – 2024-12-23	0.2615	[0.170, 0.380]	65	0	❌

Window Nov 26–Dec 2 had 52 of 65 rows (80%) suppressed by ADX — the market entered a sustained low-trend/ranging regime across those 5 days. Dec 3–9 had 26 suppressions as ADX recovered. The final two windows saw zero suppressions — full pass-through, which explains why overall accuracy in those windows is driven entirely by the pod's directional bias.

4.3 Conviction Reliability Bins

All 312 pass-through signals. Mean conviction per bin vs estimated accuracy (overall accuracy = 0.2949, flat across bins — ECE=0.5586 confirms zero calibration signal):

Conviction bin	Mean conviction	n (% of active)	Expected accuracy	Observed (≈)	Gap
0.40 – 0.60	0.54	4 (1.3%)	~54%	~0.30	0.24
0.60 – 0.75	0.70	33 (10.6%)	~70%	~0.30	0.40
0.75 – 0.90	0.81	152 (48.7%)	~81%	~0.30	0.51
0.90 – 1.00	0.96	123 (39.4%)	~96%	~0.30	0.66

Only 1.3% of signals fell below conviction 0.60. The model clusters 88% of signals in the 0.75–1.00 range. The calibration gap widens with higher conviction – the model becomes more wrong as it becomes more confident. This is the worst possible calibration pattern for a quality gate.

4.4 RAG Condition

Metric	No-RAG	With RAG
Global accuracy	0.2949	0.3742
CE / PE / NEUTRAL	234/78/78	95/106/189
Ground truth	99/140/151	same

RAG significantly improved class balance and overall accuracy. See Section 3.7.

5. How Do I Know This Pod Is Safe?

5.1 Scenario: 09:30 Expiry Thursday – VIX 3σ Above Mean, ADX = 14

This specific scenario is the most important one in this section. Walking through it precisely:

Step 1 – Market state ingestion:

The market state arrives: `vix_india = ~28.0` (3σ above 30-day mean of ~17), `adx_14 = 14`.

Step 2 – Regime check (Rule 1: ADX < 20):

`adx_14 = 14 < 20.0` → **Rule 1 fires immediately.**

```
{
  "action": "SUPPRESSED_ADX",
  "values": {"adx_14": 14.0, "threshold": 20.0},
  "final_direction": "NEUTRAL",
  "conviction": 0.0
}
```

The pod is never called. The orchestrator returns `NEUTRAL`, `conviction=0.0` and writes the above record to `orchestrator_decisions.jsonl`. No model inference happens.

Step 3 – Downstream output:

```
{"direction": "NEUTRAL", "conviction": 0.0, "horizon": "intraday",
  "signal_id": "<uuid>", "generated_at": "2024-XX-XXT09:30:00+05:30",
```

```
"orchestrator_action": "SUPPRESSED_ADX"}
```

The orchestrator correctly suppresses this signal. ADX=14 in a VIX spike is exactly the regime this rule is designed for – low ADX means no directional trend, so no options signal should pass downstream.

5.2 What Is Wrong With This Implementation

The suppression is correct in isolation. Here is what is wrong:

1. The suppression log does not capture elevated VIX context. The log records `adx_14=14` and the threshold, but not that VIX is simultaneously 3σ elevated. An operations team reviewing the log file sees a routine ADX suppression, not a stress event. Fix: log all market state features on every suppression, not just the triggering value.

2. After the VIX spike, ADX will eventually cross 20. As volatility settles, ADX tends to rise as directional price action develops. Once $ADX \geq 20$, the orchestrator will resume calling the pod – but the pod was fine-tuned on data where VIX was in a normal range. The model has never seen a post-spike recovery regime. The orchestrator has no mechanism to detect that the pod is now operating out-of-distribution.

3. No VIX-based independent circuit breaker. The current three rules do not include VIX as a direct suppression criterion. A state with $VIX=28$, $ADX=22$ (trending) would pass all three orchestrator rules and reach the pod – which has no training signal for that regime combination.

4. ADX suppression is binary with no lookback. If ADX reads 19.9 on one bar and 20.1 on the next, the orchestrator toggles on and off. There is no hysteresis or rolling confirmation window.

What I would add:

- Log full market state on every orchestrator decision
- Add VIX-based circuit breaker: if `vix_india > mean_30d * 1.5`, suppress regardless of ADX
- Add rolling ADX confirmation: require $ADX \geq 20$ for N consecutive bars before resuming
- Add a post-spike recovery flag: if VIX was above threshold in the last K bars, hold the pod suppressed for a cooldown period

5.3 What the Eval Suite May Have Missed

Training-eval regime mismatch: The eval window (days 31–60) is a different market regime than the training window (days 1–30). Training was CE-leaning ($CE=75 > PE=70$); eval shifted to PE-dominant ($PE=140 > CE=99$). The 5-day window breakdown shows accuracy as low as 0.21 in some windows – but we cannot distinguish whether this is model failure or regime shift without a reference signal.

The Nov 26–Dec 2 window (n=13, 46 suppressed) is the most important finding in the window breakdown. With 46 of ~65 rows suppressed by ADX, the market was in a sustained low-trend/high-uncertainty state – the exact scenario the pod is worst at. Of the 13 signals that passed, accuracy was 0.23. This window should be flagged for special review in a production system.

Conviction calibration was not validated during training. There was no calibration holdout set. The model was evaluated on conviction accuracy after training, not calibrated against it. A correct setup would reserve 10% of training data as a calibration set, fit Platt scaling, and apply it at inference.

5.4 Safety Property Status

Property	Designed	Implemented	Effective
Schema safety (parse + regex fallback)	✓	✓	✓ Always
Regime filter (ADX < 20)	✓	✓	✓ Always – correctly fires at ADX=14
Conviction quality gate	✓	✓	✗ Threshold never triggered; ECE=0.56
VIX-based stress suppression	Missing	✗	✗ Gap
Post-spike recovery cooldown	Missing	✗	✗ Gap
Signal diversity monitor	Missing	✗	✗ Gap

5.5 Honest Assessment

For the specific expiry–Thursday scenario: **the system responds correctly** – ADX=14 triggers immediate suppression, the pod is never called, NEUTRAL is returned with a logged reason code. This is the deterministic layer proving edge before the AI layer is consulted.

For the broader safety question: **the pod is not production-safe without RAG**, and the conviction gate is non-functional as currently calibrated. The infrastructure is correct; the signal quality layer fails due to insufficient training data. The eval suite detected this against pre-committed thresholds – which is itself the correct engineering outcome. A system that fails loudly, traceably, and against pre-stated criteria is the foundation of a safe system, not a finished one.

6. Appendix

A. Repository Structure

```

quant_singularity/
├── data_audit.py           # Pre-committed; produces clean dataset
├── eval_suite.py          # Pre-committed; defines all thresholds
├── pod.py                 # Inference + JSON parse + regex fallback
├── orchestrator.py        # 3-rule safety layer + decision log
├── finetune_local.py      # Local GPU training
├── finetune_kaggle.py     # Kaggle T4 notebook
├── kaggle_cell_A_eval.py  # Comprehensive eval (5-day, VIX regime, F
├── kaggle_cell_B_rag.py   # RAG ablation
├── run_eval.py            # Local eval runner
├── report.md / report.pdf # This report
├── slm_intern_data/
│   ├── finetune_instructions_clean.jsonl
│   ├── market_states.parquet
│   ├── rag_corpus.jsonl
│   └── retrieve.py        # Provided, unchanged
└── adapter/              # Final LoRA weights

```

B. MLflow Run Registry

Run	Epochs	Val Loss	Run ID	Status
Clean 3-epoch (final)	3	0.974	f0a11051b56d44e790bb9d1ad3aef18c	Submitted
Prior 3-epoch	3	0.969	2c8edc45779d4237a35a2629fd1fe464	Superseded
5-epoch test	5	1.151	6437e9ed98d142a6b2941217f4081583	Ablation
Balanced retrain	3	—	—	Ablation

C. Key Design Decisions

Decision	Choice	Rationale
Base model	TinyLlama-1.1B	VRAM fit, instruction-tuning, low overfitting risk at 261 samples
LoRA rank	8	Sufficient for narrow structured task, avoids memorisation
Epochs	3	Val loss plateau; epoch 4–5 showed no improvement
max_new_tokens	128	JSON ~75 tokens; prevents runaway signal_id generation

Decision	Choice	Rationale
Inference temperature	0.1	Near-deterministic; 0.3 caused 4× more JSON truncation
Conviction threshold	0.40	Per spec; recommend raising to ≥ 0.75 given ECE=0.56
RAG k	3	Top-3 episodes sufficient without exceeding prompt budget